

Arhitectura Sistemelor de Calcul

Lect. Dr. Șotropa Diana
diana.sotropa@ubbcluj.ro



Facultatea de Matematică și Informatică
Universitatea Babeș-Bolyai





Programarea multimodul

Material creat de către Lect. Dr. Vancea Alexandru împreună cu Marius Vanța
Bitdefender 2017

Tehnici și instrumente

Legarea statică la linkeditare: nasm + limbaje de nivel înalt

- Cerințe ale editorului de legături
 - Directiva global pentru permis acces din alt limbaj către etichetele noastre
 - Directiva extern pentru obținut acces în NASM către resursele implementate în alte limbaje
 - Declararea în limbajul de nivel înalt a variabilelor și subrutinelor definite în NASM
 - Exemplu C: declaratorul extern!
- Intrarea în procedură
- Nealterarea valorilor unor regiștri
- Transmiterea și accesarea parametrilor
- Alocarea de spațiu pentru datele locale (opțional)
- Întoarcerea unui rezultat (opțional)

Ultimele aspecte sunt discutate în detaliu la CONVENȚII DE APEL!

- Vezi interfațarea cu limbajele de nivel înalt: convenții de apel

Tehnici și instrumente

Programare multimodul asm+asm

Main.asm

```
bits 32
// declar procedura definita in asamblare
extern suma
segment code use32 class=code
start:
; apelul procedurii definite in asamblare
    push dword [b]
    push dword [a]
    call suma
    add esp, 2*4

    push dword 0
    call [exit]
```

offset revenire	<-- ESP
valoarea lui a	ESP+4
valoarea lui b	ESP+8
...	
...	

Modul.asm

```
bits 32
; facem vizibila procedura pentru toate modulele
global suma
segment data use32 class=data
segment code use32 class=code
    suma:
; obtin valorile argumentelor de pe stiva
    mov eax, [esp+4]           ; EAX = a
    mov edx, [esp+8]           ; EDX = b
    add eax, edx               ; EAX = a+b
    ret
```

Tehnici și instrumente

Programare multimodul asm+c

Main.c

```
#include <stdio.h>

// declar procedura definita in asamblare
int suma(int, int);

int main(void){

// apelul procedurii definite in
asamblare

    int sum = suma(a, b);
    return 0;

}
```

Cadrul de stivă al procedurii suma		
	vechea valoare din EBP	<-- ESP
Cadrul de stivă al funcției main()	offset revenire in main()	EBP+4
	valoarea lui a	EBP+8
	valoarea lui b	EBP+12
	...	
	...	

Modul.asm

```
bits 32

global _suma

segment data use32 class=data

segment code use32 class=code

    _suma:

; creez noul cadru de stiva

    push ebp

    mov ebp, esp

; obtin valorile argumentelor de pe stiva

    mov eax, [ebp+8]; EAX = a

    mov edx, [ebp+12]; EDX = b

    add eax, edx; EAX = a+b

; refac cadrul de stiva al functiei main()

    mov esp, ebp

    pop ebp

    ret
```

Tehnici și instrumente

Exemplu program multimodul asm + C

```
// AFISARE.C
```

```
// solicită către preprocesorul de C includerea fișierului stdio.h
```

```
// stdio.h declară antetul (tipul de rezultat și parametri) funcției C printf
```

```
#include <stdio.h>
```

```
// declarăm funcția din fișierul asm încât compilatorul C să cunoască tipul de parametri și rezultat
```

```
// linkeditorul se va ocupa de implementarea funcției, compilatorul necesită doar să îi cunoască antetul
```

```
void asm_start (void);
```

```
// echivalent ca efect cu extern void asm_start(void) ! Orice funcție declarată la nivelul cel mai exterior al unui modul C face parte din clasa de memorie extern
```

```
// funcția afisare este apelată de către codul asm
```

```
void afisare (int *vector, int numar_elemente){
```

```
// orice functie definită la nivelul cel mai exterior al unui modul C este implicit globală, adică se exportă implicit
```

```
    for (int index=0; index < numar_elemente; index ++)
```

```
        printf("%d", vector[index]);
```

```
    printf("\n");
```

```
}
```

```
// programul principal, acesta apelează funcția asm_start scrisă în asamblare
```

```
void main(void){ // de aici începe execuția programului final
```

```
    asm_start(); // apelăm funcția din fișierul asm
```

```
}
```

Tehnici și instrumente

Exemplu program multimodul asm + C

; VECTOR.ASM

; informăm asamblorul despre existența funcției afișare

extern _afisare ; atenție la adăugarea _ ca prefix al numelor provenite din C

; informăm asamblorul că dorim ca asm_start să fie disponibil altor unități de compilare

global _asm_start ; atenție la adăugarea _ ca prefix al numelor referite de către C!

; codul asm este dispus într-un segment public, posibil a fi partajat cu alt cod extern

segment code public code use32

_asm_start:

push dword elemente ; parametru transmis prin valoare (se urcă în stivă valoarea 5)

push dword vector ; vectorul este transmis prin referință (adresa lui exte copiată pe stivă)

call _afisare ; apelul funcției C, din nou cu prefix _

add esp, 4*2 ; afisare este o funcție C (cdecl) -> necesită ca NOI să eliberăm argumentele

ret ; revenire la codul C care ne-a apelat

; linkeditorul poate folosi segmentul public de date și pentru date din afară

segment data public data use32

vector dd 1, 2, 3, 4, 5 ; vectorul ce-l vom afișa cu rutina C

elemente equ (\$ - vector) / 4 ; constantă egală cu 5 (numărul elementelor din vector)

Tehnici și instrumente

- Exemplu program multimodul nasm + C
 - De ce _?
 - Contruire executabil:
 - Compilare / asamblare
afisare.c poate fi compilat cu orice compilator C (după preferințe) -> afisare.obj
Visual C: `cl /c afisare.c`
`nasm.exe vector.asm -fwin32 -o vector.obj`
 - Editarea legăturilor:
Se apelează orice linkeditor compatibil C, solicitând
Intrări: afisare.obj și vector.obj
leșiri: aplicație de consolă
`link vector.obj afisare.obj /OUT:afisare.exe /MACHINE:X86 /SUBSYSTEM:CONSOLE`
- Alternativ, fișierele pot fi înglobate într-o soluție Visual Studio, instruind IDE-ul:
 - Să asambleze fișierul asm: specificând de exemplu drept Pre-Build Event comanda de asamblare de mai sus
 - Să includă afisare.obj drept intrare adițională la linkeditare
 - Există extensii pentru Visual Studio care rezolvă automat și transparent problema!

Interfațarea cu limbaje de nivel înalt

- Transmiterea parametrilor prin valoare
 - Se copiază valorile octeților ce compun datele
 - Risipitor și lent când datele ocupă multă memorie
 - Din perspectiva apelantului, datele sunt constante
- Transmiterea parametrilor prin referință (adresă / pointer)
 - Se specifică adresa (și uneori dimensiunea) datelor
 - Ineficient când datele ocupă puțin
 - 32 biți în plus pentru stocat adresa + citire pointer înainte de acces la date
 - Datele pot suferi modificări
- Decizia transmitere prin valoare sau prin referință?
 - Criteriul de performanță: dimensiunea în octeți
 - Risc de a depăși memoria disponibilă? Prin referință!
 - Criteriul de accesibilitate: trebuie modificate datele?
 - Neapărat constante? Prin valoare!

Interfațarea cu limbaje de nivel înalt

Convenții de apel

- Cum transmitem parametri către subrutine?
 - Ce tipuri de parametri se pot transmite?
 - În ce ordine?
 - Câți parametri? Oricâți?
- Ce resurse sunt volatile (poate să le altereze funcția apelată)?
- Unde se găsește rezultatul?
- Ce acțiuni de curățare sunt necesare post-apel?
 - Cine este responsabil să le efectueze?
- Convenții
 - Uzuale: CDECL, STDCALL, FASTCALL
 - Rar folosite sau învechite: PASCAL, FORTAN, SYSCALL, etc.
 - Utilizator: **în asamblare toate aspectele documentate de către o convenție sunt accesibile programatorului!**

Interfațarea cu limbaje de nivel înalt

Convenții de apel – convenția C (CDECL)

- Specifică limbajului C
- Cum transmitem parametri către subrutine? Prin împingerea lor pe stivă
 - Ce tipuri de parametri se pot transmite? Orice, dar necesită extins la minim DWORD
 - În ce ordine? Dreapta către stânga, adică, invers ordinii de la declarație
 - Câți parametri? Oricâți? Da, C permite funcții cu oricâți parametri (ex. printf)
- Ce resurse sunt volatile ? EAX, ECX, EDX, Eflags
- Unde se găsește rezultatul? EAX, EDX:EAX sau ST0 (FPU)
- Ce acțiuni de curățare sunt necesare post-apel? Eliberarea argumentelor
 - Cine este responsabil să le efectueze? Apelantul!

Parametri			Resurse volatile	Rezultate	Eliberare
Stocare	Ordine	Număr			
Stivă	Inversă	Oricâți	EAX, ECX, EDX, Eflags	EAX / EDX:EAX / ST0 (FPU)	Apelant

Interfațarea cu limbaje de nivel înalt

Convenții de apel – convenția STDCALL

- Specifică sistemului de operare Windows
 - Denumită și WINAPI
 - Folosită de către bibliotecile de sistem Windows
- Foarte asemănătoare convenției CDECL
 - Diferențe:
 - Număr fix de parametri
 - Eliberarea argumentelor o face funcția apelată

Parametri			Resurse volatile	Rezultate	Eliberare
Stocare	Ordine	Număr			
Stivă	Inversă	Fix	EAX, ECX, EDX, Eflags	EAX / EDX:EAX / ST0 (FPU)	Subrutina apelată

Interfațarea cu limbaje de nivel înalt

Apelul subrutinelor

- Etape:
 1. **Cod de apel:** pregătirea și efectuarea apelului
 2. **Cod de intrare:** intrarea în procedură și pregătirea execuției
 3. **Cod de ieșire:** revenire și eliberarea resurselor ce au expirat
- Acțiunile depind în funcție de convenția de apel a subrutinei apelate, dar etapele rămân aceleași!
- Etapele sunt tratate / implementate automat în codul generat de către compilatoarele limbajelor de nivel mai înalt
 - În asamblare rămâne totul în sarcina noastră!

Interfațarea cu limbaje de nivel înalt

Apelul subrutinelor – cod de apel

- Sarcini:
 1. Salvare resurse volatile în uz: `push registru`
 2. Asigurare respectare constrângeri (ESP aliniat, DF=0, ...)
 3. Pregătire argumente (stivă, conform convenției): `push`
 4. Efectuare apel: `call`
 - `call subrutină`, când subrutină este linkeditată static
 - `call [subrutină]`, dacă este dinamică (la link-time)
 - `call registru` sau `call [variabilă]` pentru dinamică la runtime
- Subrutinele asm folosite doar din asm pot evita (din simplitate si / sau eficiență) aceste sarcini

Interfațarea cu limbaje de nivel înalt

Apelul subrutinelor – cod de apel

Exemplu: apel printf din asm pentru afișarea numere 0 ... 9

```
import exit msvcrt.dll
import printf msvcrt.dll
extern exit, printf
global start
```

```
segment code use32
```

```
start:
```

```
    mov ecx, 10
    xor eax, eax
```

```
.next:
```

```
    push eax
    push ecx
```

```
    push eax
    push dword format_string
    call [printf]
    add esp, 2*4
```

```
    pop ecx
    pop eax
    inc eax
```

```
    loop .next
    push dword 0
    call [exit]
```

```
segment data use32
```

```
    format_string db "%d", 10, 13, 0
```

Dacă apelul se făcea din C, compilatorul ar fi generat singur codul de apel!
Cum apelul se face din asamblare, codul de apel trebuie scris de către noi!

(1) Salvare resurse volatile

(2) DF=0, stiva a fost folosită doar pe DWORD

(3) Pregătire argumente apel CDECL

(4) Efectuare apel

Restaurare resurse volatile (apelantul)

Interfațarea cu limbaje de nivel înalt

Apelul subrutinelor – cod de apel

- Efectul codului de apel asupra stivei

```
push eax
push ecx
push eax
push dword format_string
call [printf]
add esp, 2*4
```

Stare inițială

ESP →

Date program apelant

Interfațarea cu limbaje de nivel înalt

Apelul subrutinelor – cod de apel

- Efectul codului de apel asupra stivei

```
push eax  
push ecx
```



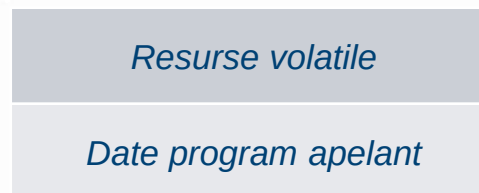
(1) Salvare resurse volatile

```
push eax  
push dword format_string  
call [printf]  
add esp, 2*4
```

(1) Salvare resurse volatile

ESP →

Adrese mari →



Interfațarea cu limbaje de nivel înalt

Apelul subrutinelor – cod de apel

- Efectul codului de apel asupra stivei

```
push eax  
push ecx
```

```
push eax  
push dword format_string  
call [printf]  
add esp, 2*4
```

(1) Salvare resurse volatile

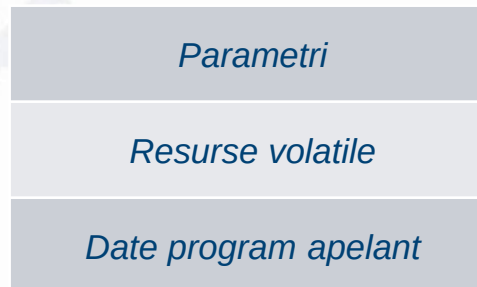
(2) DF=0, stiva a fost folosită doar pe DWORD

(3) Pregătire argumente apel CDECL

(3) Pregătire argumente apel CDECL

ESP

Adrese mari



Interfațarea cu limbaje de nivel înalt

Apelul subrutinelor – cod de apel

- Efectul codului de apel asupra stivei

```
push eax  
push ecx
```

```
push eax
```

```
push dword format_string
```

```
call [printf]
```

```
add esp, 2*4
```

(1) Salvare resurse volatile

(2) DF=0, stiva a fost folosită doar pe DWORD

(3) Pregătire argumente apel CDECL

(4) Efectuare apel

(4) Efectuare apel

ESP

Adresă Revenire

Parametri

Resurse volatile

Date program apelant

Adrese mari

Interfațarea cu limbaje de nivel înalt

Apelul subrutinelor – cod de apel

APELANTUL

- a) Salvare resurse volatile (EAX, ECX, EDX, EFLAGS)
- b) Transmitere parametri
- c) Efectuare apel cu salvare adresa de revenire

Interfațarea cu limbaje de nivel înalt

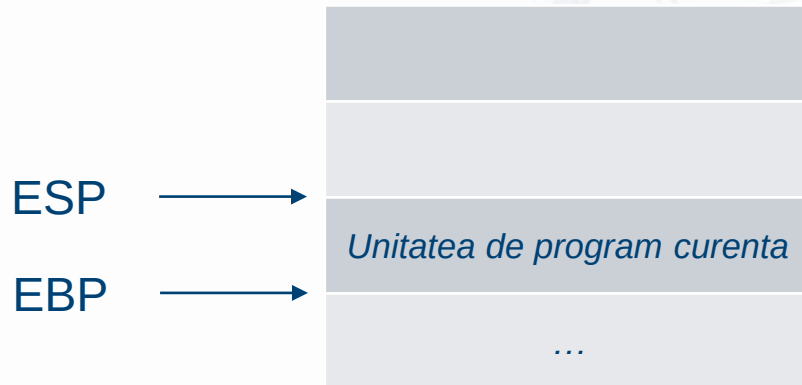
Apelul subrutinelor – cod de intrare

- Sarcini:
 1. Configurarea unui **cadru de stivă (stack frame)**: reper EBP sau ESP?
 2. Pregătire variabile locale ale funcției: SUB ESP, număr octeți
 3. Salvarea unei copii a resurselor nevolatile modificate: PUSH registru
 - Orice regiștri cu excepția celor volatili
- **Cadru de stivă**: structură de date stocată în stivă, de dimensiune fixă (pentru o subrutină dată) și conținând:
 - Parametrii pregătiți de apelant
 - Adresa de revenire (către instrucțiunea ce-i urmează celei de apel)
 - Copii ale resurselor nevolatile folosite de subrutină
 - Variabile locale
- Subrutinele asm folosite doar din asm pot evita (din simplitate si / sau eficiență) aceste sarcini

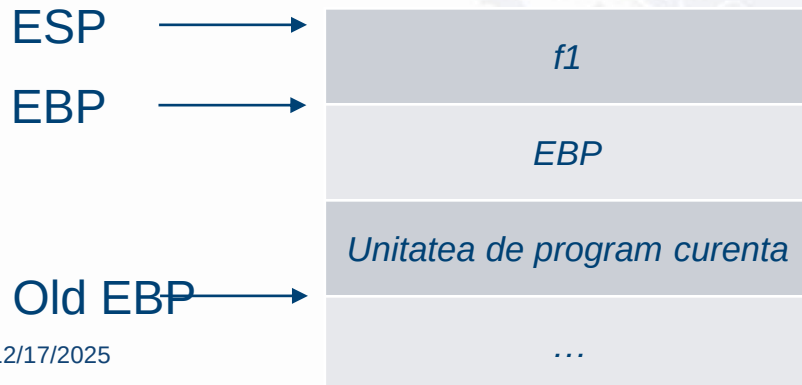
Interfațarea cu limbaje de nivel înalt

Apelul subrutinelor – cod de intrare

- Cadrul de stiva: UPC



- Cadrul de stiva: se apelează o funcție f1



Interfațarea cu limbaje de nivel înalt

Apelul subrutinelor – cod de intrare

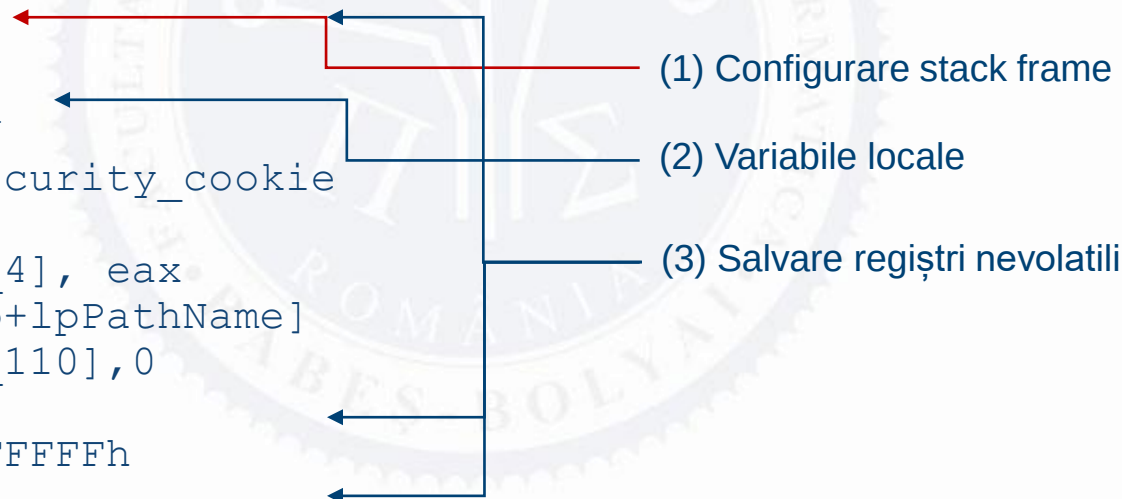
Exemplu: cod intrare în funcția CDECL `chdir`, generată de compilatorul C

```
; dezasamblat cu IDA (The Interactive Disassembler)
; sintaxa corespunde asamblorului MASM
; int __cdecl chdir (const char *)
```

`chdir:`

```
mov edi, edi
push ebp
mov ebp, esp
sub esp, 118h
```

```
mov eax, __security_cookie
xor eax, ebp
mov [ebp+var_4], eax
mov eax, [ebp+lpPathName]
and [ebp+var_110], 0
push ebx
or ebx, 0FFFFFFFFh
push esi
```



```
; ...
```

Codul de intrare a fost generat automat de către compilator

Interfațarea cu limbaje de nivel înalt

Apelul subrutinelor – cod de intrare

Exemplu: cod intrare într-o funcție STDCALL scrisă în asm de către noi

factorial: ; în modulul apelant C se declară **extern int _stdcall factorial (int n)**

```
push ebp
mov ebp, esp ; definim un cadru de stivă cu EBP pivot / referință
sub esp, 4 ; alocăm spațiu pe stivă pentru o variabila DWORD temporară
push ebx ; salvăm ebx pentru a-l putea restaura la sfârșit

mov eax, [ebp+8] ; încărcăm valoarea argumentului din stackframe (n-ul curent)

cmp eax, 2 ; testăm condiția de terminare (n<2)
jae .recursive
mov eax, 1 ; returnăm 1 când n < 2
jmp .gata

.recursiv:
push eax ; reținem valoarea lui n să nu-l pierdem la apelul recursiv
dec eax ; pregătim noul parametru n-1 pt a apela factorial(n-1)
push eax ; urcăm în stiva valoarea n-i ca parametru pt factorial (n-1)
call factorial ; apel recursiv (STDCALL)

mov [ebp-4], eax ; salvăm în DWORD-ul temporar valoarea lui factorial (n-1)
pop eax ; restaurăm valoarea lui n
mov ebx, [ebp-4] ; ebx <- factorial(n-1) preluat din variabila temporară
mul ebx ; calculăm edx:eax = (EAX=) n * (EBX=)factorial(n-1)

.gata:
pop ebx ; refacem EBX la valoarea inițială
add esp, 4 ; eliberăm memoria ocupată de variabila temporară
pop ebp ; restaurăm ebp la valoarea inițială
ret 4 ; STDCALL: revenire cu eliberare memorie parametri
```

(1) Configurare stack frame
(2) Variabile locale
(3) Salvare regiștri nevolatili

```
int factorial(int n){
    if(n==1) return 1;
    else return n*factorial(n-1);
}
```

Interfațarea cu limbaje de nivel înalt

Apelul subrutinelor – cod de intrare

Exemplu: cod intrare într-o funcție STDCALL scrisă în asm de către noi - STACKFRAME

factorial: ; în modulul apelant C se declară **extern int _stdcall factorial (int n)**

```
➔ push ebp
mov ebp, esp ; definim un cadru de stivă cu EBP pivot / referință
sub esp, 4 ; alocăm spațiu pe stivă pentru o variabila DWORD temporară
push ebx ; salvăm ebx pentru a-l putea restaura la sfârșit

mov eax, [ebp+8] ; încercăm valoarea argumentului din stackframe (n-ul curent)

cmp eax, 2 ; testăm condiția de terminare (n<2)
jae .recursive
mov eax, 1 ; returnăm 1 când n < 2
jmp .gata

.recursiv:
push eax ; reținem valoarea lui n să nu-l pierdem la apelul recursiv
dec eax ; pregătim noul parametru n-1 pt a apela factorial(n-1)
push eax ; urcăm în stiva valoarea n-i ca parametru pt factorial (n-1)
call factorial ; apel recursiv (STDCALL)

mov [ebp-4], eax ; salvăm în DWORD-ul temporar valoarea lui factorial (n-1)
pop eax ; restaurăm valoarea lui n
mov ebx, [ebp-4] ; ebx <- factorial(n-1) preluat din variabila temporară
mul ebx ; calculăm edx:eax = (EAX=) n * (EBX=)factorial(n-1)

.gata:
pop ebx ; refacem EBX la valoarea inițială
add esp, 4 ; eliberăm memoria ocupată de variabila temporară
pop ebp ; restaurăm ebp la valoarea inițială
ret 4 ; STDCALL: revenire cu eliberare memorie parametri
```

Stare la intrare

ESP

Adresă Revenire

Parametri

Resurse volatile

Date program apelant

Adrese mari

Interfațarea cu limbaje de nivel înalt

Apelul subrutinelor – cod de intrare

Exemplu: cod intrare într-o funcție STDCALL scrisă în asm de către noi - STACKFRAME

factorial: ; în modulul apelant C se declară **extern int _stdcall factorial (int n)**

push ebp

mov ebp, esp ; definim un cadru de stivă cu EBP pivot / referință

sub esp, 4 ; alocăm spațiu pe stivă pentru o variabila DWORD temporară

push ebx ; salvăm ebx pentru a-l putea restaura la sfârșit

mov eax, [ebp+8] ; încercăm valoarea argumentului din stackframe (n-ul curent)

cmp eax, 2 ; testăm condiția de terminare (n<2)

jae .recursive

mov eax, 1 ; returnăm 1 când n < 2

jmp .gata

.recursive:

push eax ; reținem valoarea lui n să nu-l pierdem la apelul recursiv

dec eax ; pregătim noul parametru n-1 pt a apela factorial(n-1)

push eax ; urcăm în stiva valoarea n-i ca parametru pt factorial (n-1)

call factorial ; apel recursiv (STDCALL)

mov [ebp-4], eax ; salvăm în DWORD-ul temporar valoarea lui factorial (n-1)

pop eax ; restaurăm valoarea lui n

mov ebx, [ebp-4] ; ebx <- factorial(n-1) preluat din variabila temporară

mul ebx ; calculăm edx:eax = (EAX=) n * (EBX=)factorial(n-1)

.gata:

pop ebx ; refacem EBX la valoarea inițială

add esp, 4 ; eliberăm memoria ocupată de variabila temporară

pop ebp ; restaurăm ebp la valoarea inițială

ret 4 ; STDCALL: revenire cu eliberare memorie parametri

(1) Configurare stack frame

(1) Configurare stack frame

ESP, EBP

EBP apelant

Adresă Revenire

Parametri

Resurse volatile

Date program apelant

26

Adrese mari

Interfațarea cu limbaje de nivel înalt

Apelul subrutinelor – cod de intrare

Exemplu: cod intrare într-o funcție STDCALL scrisă în asm de către noi - STACKFRAME

factorial: ; în modulul apelant C se declară **extern int _stdcall factorial (int n)**

```
push ebp
mov ebp, esp ; definim un cadru de stivă cu EBP pivot / referință
sub esp, 4 ; alocăm spațiu pe stivă pentru o variabila DWORD temporară
push ebx ; salvăm ebx pentru a-l putea restaura la sfârșit
mov eax, [ebp+8] ; încercăm valoarea argumentului din stackframe (n-ul curent)
```

```
cmp eax, 2 ; testăm condiția de terminare (n<2)
jae .recursive
mov eax, 1 ; returnăm 1 când n < 2
jmp .gata
```

.recursive:

```
push eax ; reținem valoarea lui n să nu-l pierdem la apelul recursiv
dec eax ; pregătim noul parametru n-1 pt a apela factorial(n-1)
push eax ; urcăm în stiva valoarea n-i ca parametru pt factorial (n-1)
call factorial ; apel recursiv (STDCALL)
```

```
mov [ebp-4], eax ; salvăm în DWORD-ul temporar valoarea lui factorial (n-1)
pop eax ; restaurăm valoarea lui n
mov ebx, [ebp-4] ; ebx <- factorial(n-1) preluat din variabila temporară
mul ebx ; calculăm edx:eax = (EAX=) n * (EBX=)factorial(n-1)
```

.gata:

```
pop ebx ; refacem EBX la valoarea inițială
add esp, 4 ; eliberăm memoria ocupată de variabila temporară
pop ebp ; restaurăm ebp la valoarea inițială
ret 4 ; STDCALL: revenire cu eliberare memorie parametri
```

(1) Configurare stack frame

(2) Variabile locale

(2) Variabile locale

ESP

EBP

Variabile locale

EBP apelant

Adresă Revenire

Parametri

Resurse volatile

Date program apelant 27

Adrese mari

Interfațarea cu limbaje de nivel înalt

Apelul subrutinelor – cod de intrare

Exemplu: cod intrare într-o funcție STDCALL scrisă în asm de către noi - STACKFRAME

factorial: ; în modulul apelant C se declară **extern int _stdcall factorial (int n)**

```
push ebp ←  
mov ebp, esp ; definim un cadru de stivă cu EBP pivot / referință  
sub esp, 4 ; alocăm spațiu pe stivă pentru o variabila DWORD temporară  
push ebx ; salvăm ebx pentru a-l putea restaura la sfârșit
```

(1) Configurare stack frame

→ mov eax, [ebp+8] ; încercăm valoarea argumentului din stackframe (n-ul curent)

(2) Variabile locale

```
cmp eax, 2 ; testăm condiția de terminare (n<2)  
jae .recursive  
mov eax, 1 ; returnăm 1 când n < 2  
jmp .gata
```

(3) Salvare regiștri nevolatili

.recursive:

ESP

```
push eax ; reținem valoarea lui n să nu-l pierdem la apelul recursiv  
; Aici se generează codul de apel pentru următorul apel recursiv  
dec eax ; pregătim noul parametru n-1 pt a apela factorial(n-1)  
push eax ; urcăm în stiva valoarea n-i ca parametru pt factorial (n-1)  
call factorial ; apel recursiv (STDCALL)
```

EBP

```
mov [ebp-4], eax ; salvăm în DWORD-ul temporar valoarea lui factorial (n-1)  
pop eax ; restaurăm valoarea lui n  
mov ebx, [ebp-4] ; ebx <- factorial(n-1) preluat din variabila temporară  
mul ebx ; calculăm edx:eax = (EAX=) n * (EBX=)factorial(n-1)
```

.gata:

```
pop ebx ; refacem EBX la valoarea inițială  
add esp, 4 ; eliberăm memoria ocupată de variabila temporară  
pop ebp ; restaurăm ebp la valoarea inițială  
ret 4 ; STDCALL: revenire cu eliberare memorie parametri
```

Regiștri nevolatili

Variabile locale

EBP apelant

Adresă Revenire

Parametri

Resurse volatile

Date program apelant 28

Adrese mari

Interfațarea cu limbaje de nivel înalt

Apelul subrutinelor – cod de intrare

Exemplu: cod intrare într-o funcție STDCALL scrisă în asm de către noi - STACKFRAME

factorial: ; în modulul apelant C se declară **extern int _stdcall factorial (int n)**

```
push ebp
mov ebp, esp ; definim un cadru de stivă cu EBP pivot / referință
sub esp, 4 ; alocăm spațiu pe stivă pentru o variabila DWORD temporară
push ebx ; salvăm ebx pentru a-l putea restaura la sfârșit
```

➔ `mov eax, [ebp+8];` încercăm valoarea argumentului din stackframe (n-ul curent)

```
cmp eax, 2 ; testăm condiția de terminare (n<2)
jae .recursive
mov eax, 1 ; returnăm 1 când n < 2
jmp .gata
```

.recursive:

```
push eax ; reținem valoarea lui n să nu-l pierdem la apelul recursiv
;Aici se generează codul de apel pentru următorul apel recursiv
dec eax ; pregătim noul parametru n-1 pt a apela factorial(n-1)
push eax ; urcăm în stiva valoarea n-i ca parametru pt factorial (n-1)
call factorial ; apel recursiv (STDCALL)
```

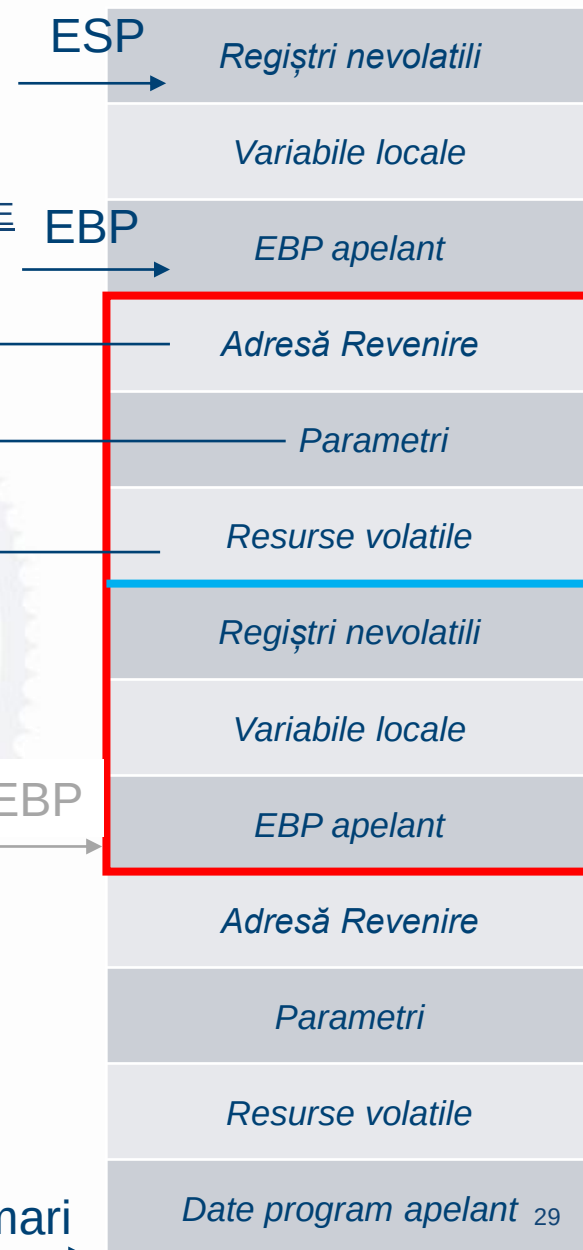
➔ `mov [ebp-4], eax;` salvăm în DWORD-ul temporar valoarea lui factorial (n-1)

```
pop eax ; restaurăm valoarea lui n
mov ebx, [ebp-4] ; ebx <- factorial(n-1) preluat din variabila temporară
mul ebx ; calculăm edx:eax = (EAX=) n * (EBX=)factorial(n-1)
```

.gata:

```
pop ebx ; refacem EBX la valoarea inițială
add esp, 4 ; eliberăm memoria ocupată de variabila temporară
pop ebp ; restaurăm ebp la valoarea inițială
ret 4 ; STDCALL: revenire cu eliberare memorie parametri
```

Adrese mari



Interfațarea cu limbaje de nivel înalt

Apelul subrutinelor – cod de intrare

a) Creare stackframe nou

- Codul de intrare implica crearea unui nou cadru de stivă pentru subrutina apelată:

```
PUSH EBP ; pentru restaurarea bazei cadrului de stivă  
          ; curent după revenire
```

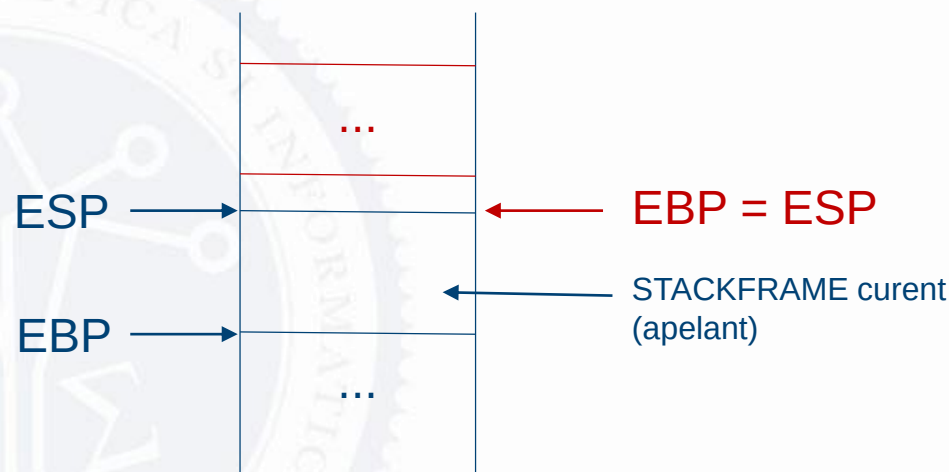
```
MOV EBP, ESP ; aici are loc formarea noului cadru de  
              ; stivă (sizeof initial = 0)
```

```
          ; ESP va descrește pe măsură ce o să  
          ; aibă loc operații de tip PUSH pe stivă
```

b) Alocare spațiu variabile locale

```
SUB ESP, nr_octeti
```

c) Salvare resurse nevolatile posibil a fi modificate



Interfațarea cu limbaje de nivel înalt

Apelul subrutinelor – cod de ieșire

- Sarcini:
 1. Restaurare resurse nevolatile alterate
 2. Eliberarea variabilelor locale ale funcției
 3. Dezafectarea cadrului de stivă
 4. Revenirea din funcție și eliberarea argumentelor
 - CDECL:
 - Subrutina apelată: ret
 - Subprogramul apelant: add esp, dimensiune_argumente
 - STDCALL:
 - ret dimensiune_argumente
- Exceptând resursele volatile și rezultatele directe ale funcției, starea programului după acești pași trebuie să reflecte starea inițială, de dinainte de apel!

Interfațarea cu limbaje de nivel înalt

Apelul subrutinelor – cod de ieșire

Exemplu: cod ieșire dintr-o funcție STDCALL asm (din apel recursiv)- STACKFRAME

factorial:

```
push ebp
mov ebp, esp
sub esp, 4
push ebx
```

```
mov eax, [ebp+8]
```

```
cmp eax, 2
jae .recursive
mov eax, 1
jmp .gata
```

.recursive:

```
push eax
dec eax
push eax
call factorial
```

```
mov [ebp-4], eax
pop eax
mov ebx, [ebp-4]
mul ebx
```

.gata:

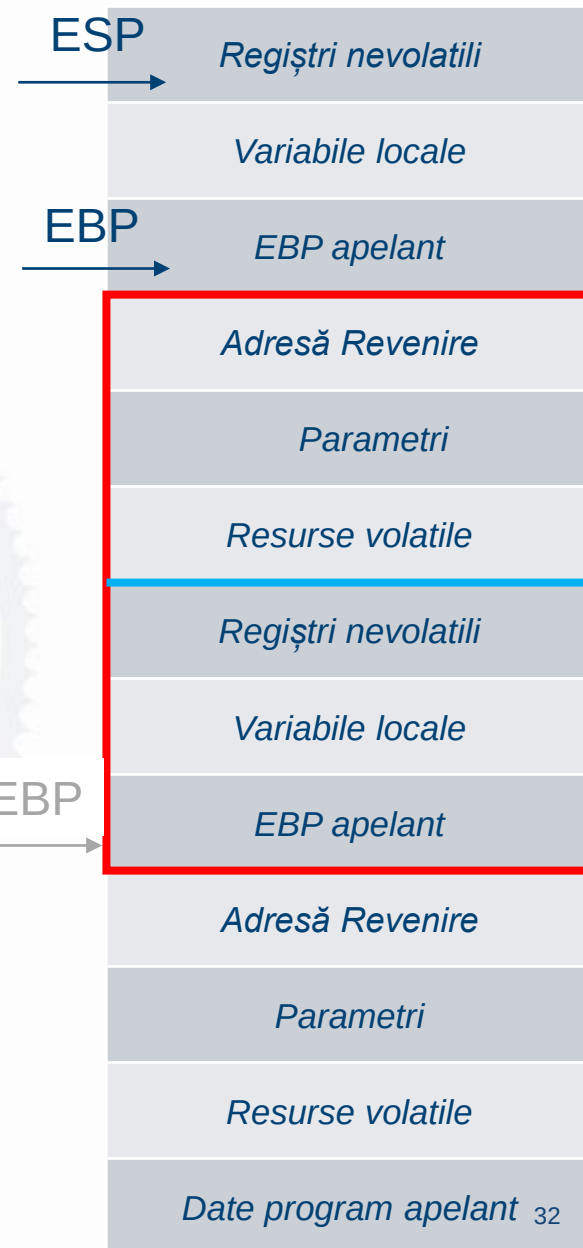
```
pop ebx
add esp, 4
pop ebp
ret 4
```

(1) Regiștri nevolatili

(2) Eliberare variabile

(3) Dezafectare stackframe

(4) Revenire și eliberare



Apelul subrutinelor – cod de ieșire

factorial:

```
pop ebx
add esp, 4
pop ebp
ret 4
```

(1) Regiștri nevolatili



Interfațarea cu limbaje de nivel înalt

Apelul subrutinelor – cod de ieșire

Exemplu: cod ieșire dintr-o funcție STDCALL asm (din apel recursiv)- STACKFRAME

factorial:

```
push ebp
mov ebp, esp
sub esp, 4
push ebx
```

```
mov eax, [ebp+8]
```

```
cmp eax, 2
jae .recursive
mov eax, 1
jmp .gata
```

.recursive:

```
push eax
dec eax
push eax
call factorial
```

```
mov [ebp-4], eax
pop eax
mov ebx, [ebp-4]
mul ebx
```

.gata:

```
pop ebx
add esp, 4
pop ebp
ret 4
```

(2) Eliberare variabile



Interfațarea cu limbaje de nivel înalt

Apelul subrutinelor – cod de ieșire

Exemplu: cod ieșire dintr-o funcție STDCALL asm (din apel recursiv)- STACKFRAME

factorial:

```
push ebp
mov ebp, esp
sub esp, 4
push ebx

mov eax, [ebp+8]

cmp eax, 2
jae .recursiv
mov eax, 1
jmp .gata

.recursiv:
push eax
dec eax
push eax
call factorial

mov [ebp-4], eax
pop eax
mov ebx, [ebp-4]
mul ebx

.gata:
pop ebx
add esp, 4
pop ebp
ret 4
```

ESP



EBP



Adresă Revenire

Parametri

Resurse volatile

Regiștri nevolatili

Variabile locale

EBP apelant

Adresă Revenire

Parametri

Resurse volatile

Date program apelant 35

(3) Dezafectare stackframe



Interfațarea cu limbaje de nivel înalt

Apelul subrutinelor – cod de ieșire

Exemplu: cod ieșire dintr-o funcție STDCALL asm (din apel recursiv)- STACKFRAME

factorial:

```
push ebp
mov ebp, esp
sub esp, 4
push ebx
```

```
mov eax, [ebp+8]
```

```
cmp eax, 2
jae .recursiv
mov eax, 1
jmp .gata
```

.recursiv:

```
push eax
dec eax
push eax
call factorial
```

```
mov [ebp-4], eax
pop eax
mov ebx, [ebp-4]
mul ebx
```

.gata:

```
pop ebx
add esp, 4
pop ebp
ret 4
```

ESP

EBP

Resurse volatile

Regiștri nevolatili

Variabile locale

EBP apelant

Adresă Revenire

Parametri

Resurse volatile

Date program apelant 44

(4) Revenire și eliberare

Interfațarea cu limbaje de nivel înalt

Apelul subrutinelor – cod de ieșire

Exemplu: cod ieșire dintr-o funcție STDCALL asm (din apel recursiv)- STACKFRAME

factorial:

```
push ebp
mov ebp, esp
sub esp, 4
push ebx

mov eax, [ebp+8]

cmp eax, 2
jae .recursiv
mov eax, 1
jmp .gata

.recursiv:
push eax
dec eax
push eax
call factorial

mov [ebp-4], eax
pop eax
mov ebx, [ebp-4]
mul ebx

.gata:
pop ebx
add esp, 4
pop ebp
ret 4
```



ESP

EBP

Resurse volatile

Regiștri nevolatili

Variabile locale

EBP apelant

Adresă Revenire

Parametri

Resurse volatile

Date program apelant 44

Interfațarea cu limbaje de nivel înalt

Apelul subrutinelor – cod de ieșire

Exemplu: cod ieșire dintr-o funcție STDCALL asm (din apel recursiv)- STACKFRAME

factorial:

```
push ebp
mov ebp, esp
sub esp, 4
push ebx

mov eax, [ebp+8]

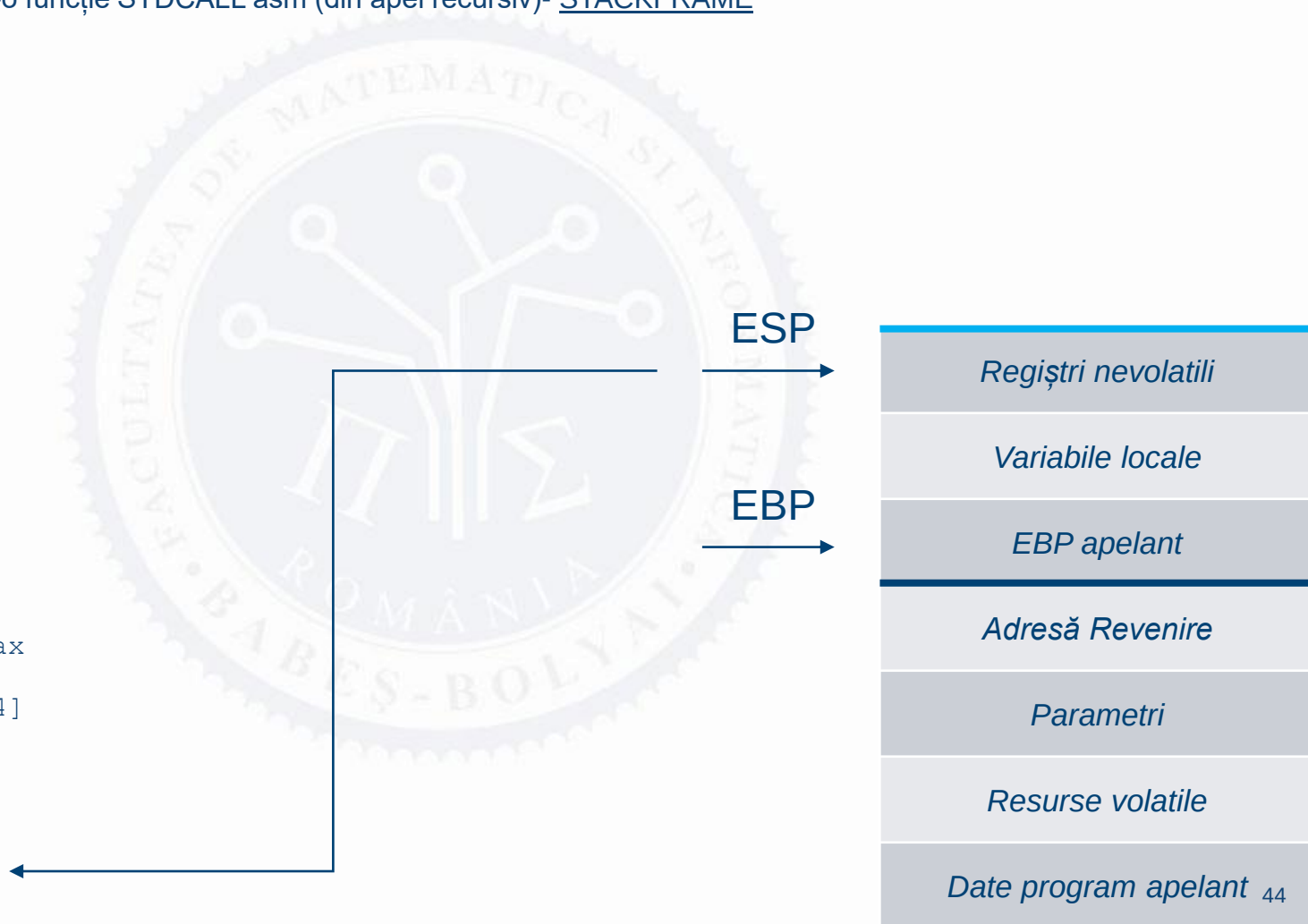
cmp eax, 2
jae .recursiv
mov eax, 1
jmp .gata

.recursiv:
push eax
dec eax
push eax
call factorial

mov [ebp-4], eax
pop eax
mov ebx, [ebp-4]
mul ebx
```

.gata:

→ pop ebx
add esp, 4
pop ebp
ret 4



Interfațarea cu limbaje de nivel înalt

Apelul subrutinelor – cod de ieșire

Exemplu: cod ieșire dintr-o funcție STDCALL asm (din apel recursiv)- STACKFRAME

factorial:

```
push ebp
mov ebp, esp
sub esp, 4
push ebx

mov eax, [ebp+8]

cmp eax, 2
jae .recursive
mov eax, 1
jmp .gata

.recursiv:
push eax
dec eax
push eax
call factorial

mov [ebp-4], eax
pop eax
mov ebx, [ebp-4]
mul ebx

.gata:
pop ebx
add esp, 4
pop ebp
ret 4
```

ESP



Resurse volatile

Date program apelant

Interfațarea cu limbaje de nivel înalt

Apelul subrutinelor – cod de ieșire

APELATUL

- a) Restaurare regiștri nevolatili
- b) Eliberare spațiu variabile locale [ADD ESP, nr octeti var locale] – menționată aici doar ca revers al pct b) din codul de intrare, dar neobligatorie deoarece eliberarea cadrului de stivă (mov esp, ebp) include oricum dpdv practic și această etapă.
- c) Eliberare cadru de stivă
MOV ESP, EBP (daca stim exact dimensiunea cadrului de stiva, ADD ESP, sizeof(stackframe) rezolva in mod similar...)
POP EBP (a, b c – reversul codului de intrare)
- d) Revenirea din subprogram (RET) și scoaterea de pe stivă a parametrilor (daca este de tip STDCALL) - (reversul b + c din codul de apel)

A mai rămas de efectuat reversul pct a) din codul de apel. Este sarcina APELATORULUI să o facă alaturi de eventuala scoatere de pe stivă a parametrilor (daca este de tip CDECL).

Aceste actiuni trebuie sa se efectueze de catre codul apelant imediat dupa incheierea apelului.

Interfațarea cu limbaje de nivel înalt

Resurse volatile vs. resurse nevolatile

- resursele **volatile** sunt reprezentate de către acei registri pe care **convenția de apel îi definește ca aparținând subrutinei apelate**, ca atare apelantul având datoria **ca parte a codului de apel** să salveze valoarea acestora (dacă-i folosește) iar apoi să le restaureze la valorile vechi la ieșirea din apel.
- *cine salvează resursele volatile ? **Apelantul** (ca parte a codului de apel).*
- *cine restaurează resursele volatile ? Tot **apelantul**, dar NU ca parte a vreunui cod de apel/intrare/ieșire, ci doar le restaurează după apel în cadrul codului curent...*
- resursele **nevolatile** sunt orice adrese de memorie ori registri care nu îi aparțin explicit subrutinei chemate, iar ca urmare, dacă aceasta dorește să efectueze modificări este necesar să le salveze la intrare (ca parte a codului de intrare) iar apoi să le restaureze înainte de ieșire la valorile lor originale (ca parte a codului de ieșire).
- Diferența apare în privința cui are obligația să facă salvarea și restaurarea:
 - resurse **volatile**: **apelantul** are datoria să aibă grijă de valorile registrilor în cauză, dacă-i folosește (subrutina chemată nu garantează nimic)
 - resurse **nevolatile**: apelantul nu are nicio datorie și îi este garantat că valorile se pastrează -- **codul apelat** este responsabil să restaureze la sfârșit orice modifica

Interfațarea cu limbaje de nivel înalt

Responsabilitati generare cod apel – cod intrare – cod iesire

		Apel fctie/proc	{	}
Apelant	Apelat	Cod apel	Cod intrare	Cod iesire
C	C	Compilerul C	Compilerul C	Compilerul C
C	asm	Compilerul C	Programator asc	Programator asc
asm	C	Programator asc	Compilerul C	Compilerul C
asm	asm	Call (salvarea adresei de revenire)	NIMIC (totul ramane la latitudinea programatorului)	RET (recuperarea adresei de revenire + salt)





FACULTATEA DE MATEMATICĂ ȘI INFORMATICĂ UNIVERSITATEA BABEȘ-BOLYAI

Str. Mihail Kogălniceanu nr. 1
Cluj-Napoca, Cluj, România

www.cs.ubbcluj.ro